



Department of Computer Science  
Faculty of Computing  
UNIVERSITI TEKNOLOGI MALAYSIA

|                                         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |                  |                  |                    |                   |
|-----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------|------------------|--------------------|-------------------|
| <b>SUBJECT NAME:</b>                    | <b>COMPUTER ORGANIZATION AND ARCHITECTURE</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |                  |                  |                    |                   |
| <b>SUBJECT CODE:</b>                    | <b>SECR 1033</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |                  |                  |                    |                   |
| <b>SEMESTER:</b>                        | <b>2 – 2023/2024</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |                  |                  |                    |                   |
| <b>LAB TITLE:</b>                       | <b>Lab 2: Arithmetic Equations &amp; Operations</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |                  |                  |                    |                   |
| <b>STUDENT INFO :</b>                   | <p>Execute the lab in group of two.</p> <table><tr><td><b>Student 1</b></td><td><b>Student 2</b></td></tr><tr><td><i>No. 1, 3, 5</i></td><td><i>No 2, 4, 6</i></td></tr></table> <p><b>Name 1:      Lim Yu Han</b></p> <p><b>Metric No:    A23CS0241</b></p> <p><b>Link for Video Demo:</b> <a href="https://youtu.be/ncTkU64Lc2E">https://youtu.be/ncTkU64Lc2E</a></p> <p><b>Name 2: Evelyn Goh Yuan Qi</b></p> <p><b>Metric No: A23CS0222</b></p> <p><b>Link for Video Demo:</b> <a href="https://youtu.be/Xx1n2WfOlCw">https://youtu.be/Xx1n2WfOlCw</a></p> | <b>Student 1</b> | <b>Student 2</b> | <i>No. 1, 3, 5</i> | <i>No 2, 4, 6</i> |
| <b>Student 1</b>                        | <b>Student 2</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |                  |                  |                    |                   |
| <i>No. 1, 3, 5</i>                      | <i>No 2, 4, 6</i>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |                  |                  |                    |                   |
| <b>SUBMISSION DATE<br/>&amp; ITEMS:</b> | <p><b>Duration of submission :-</b></p> <p>2 weeks</p> <p><b>Submission items in elearning:-</b></p> <p>1. Lab 2 exercise sheet/file (in .pdf), with the links for demo video 5 - 10min, on the cover page.</p> <p>2. The assembly programs (in .asm).</p>                                                                                                                                                                                                                                                                                                     |                  |                  |                    |                   |

**MARKS:**

\_\_\_\_\_

## Arithmetic Equation Coding in Assembly Language

Q1. Execute the program below. Determine output of the program by inspecting the content of the related registers.

a) Fill in Table 1 with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.

b) Paste the screenshot of all registers' content after each LINE is executed.

```
INCLUDE Irvine32.inc
.data
var1 word 1          ; 0001h
var2 word 9          ; 0009h

.code
main PROC
    mov ax, var1      ; LINE1
    mov bx, var2      ; LINE2
    xchg ax, bx       ; LINE3
    mov var1, ax      ; LINE4
    mov var2, bx      ; LINE5
    call DumpRegs
    exit
main ENDP
END main
```

### Answer Q1

a) Fill (Write) in the contents for the related register in each line:

|       |                                          |                                                  |
|-------|------------------------------------------|--------------------------------------------------|
| LINE1 | <b>AX = 0001h</b><br><b>var1 = 0001h</b> | Move the value of var1 (1d) into register AX     |
| LINE2 | <b>BX = 0009h</b><br><b>var2 = 0009h</b> | Move the value of var2 (9d) into register BX     |
| LINE3 | <b>AX = 0009h</b><br><b>BX = 0001h</b>   | Exchange the value of register AX and BX         |
| LINE4 | <b>AX = 0009h</b><br><b>var1 = 0009h</b> | Move the value of register AX into variable var1 |
| LINE5 | <b>BX = 0001h</b><br><b>var2 = 0001h</b> | Move the value of register BX into variable var2 |

b) Paste here screenshot of all registers' content after each LINE is executed:

LINE1:

| Memory 1                                                                                                                                                 | Registers | Autos | Watch 1 |
|----------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|-------|---------|
| EAX = 00790001 EBX = 009FF000 ECX = 005A10AA EDX = 005A10AA ESI = 005A10AA<br>EDI = 005A10AA EIP = 005A3666 ESP = 0079FAA0 EBP = 0079FAAC EFL = 00000246 |           |       |         |

LINE2:

| Memory 1                                                                                                                                                 | Registers | Autos | Watch 1 |
|----------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|-------|---------|
| EAX = 00790001 EBX = 009F0009 ECX = 005A10AA EDX = 005A10AA ESI = 005A10AA<br>EDI = 005A10AA EIP = 005A366D ESP = 0079FAA0 EBP = 0079FAAC EFL = 00000246 |           |       |         |

LINE3:

| Memory 1                                                                                                                                                 | Registers | Autos | Watch 1 |
|----------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|-------|---------|
| EAX = 00790009 EBX = 009F0001 ECX = 005A10AA EDX = 005A10AA ESI = 005A10AA<br>EDI = 005A10AA EIP = 005A366F ESP = 0079FAA0 EBP = 0079FAAC EFL = 00000246 |           |       |         |

LINE4:

| Memory 1                                                                                                                                                 | Registers | Autos | Watch 1 |
|----------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|-------|---------|
| EAX = 00790009 EBX = 009F0001 ECX = 005A10AA EDX = 005A10AA ESI = 005A10AA<br>EDI = 005A10AA EIP = 005A3675 ESP = 0079FAA0 EBP = 0079FAAC EFL = 00000246 |           |       |         |

LINE5:

| Memory 1                                                                                                                                                 | Registers | Autos | Watch 1 |
|----------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|-------|---------|
| EAX = 00790009 EBX = 009F0001 ECX = 005A10AA EDX = 005A10AA ESI = 005A10AA<br>EDI = 005A10AA EIP = 005A367C ESP = 0079FAA0 EBP = 0079FAAC EFL = 00000246 |           |       |         |

Q2. Execute the program below. Determine output of the program by inspecting the content of the related registers and watches.

- a) Fill in Table 2 with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.
- b) Paste the screenshot of all registers' content after each LINE is executed.

**Arithmetic expression:  $Rval = (-Xval + (Yval - Zval)) + 1$**

```
include irvine32.inc

.data
Rval DWORD ?
Xval DWORD 26
Yval DWORD 30
Zval DWORD 40

.code
main proc
    mov eax,Xval ; LINE1
    neg eax      ; LINE2
    mov ebx,Yval ; LINE3
    sub ebx,Zval ; LINE4
    add eax,ebx  ; LINE5
    inc eax      ; LINE6
    mov Rval,eax ; LINE7
    exit
main endp
end main
```

### **Answer Q2**

- a) Fill (Write) in the contents for the related register in each line:

|              |                                             |                                                            |
|--------------|---------------------------------------------|------------------------------------------------------------|
| <b>LINE1</b> | <b>EAX= 0000001Ah<br/>Xval= 0000001Ah</b>   | <b>Move the value of Xval (26d) into the register EAX</b>  |
| <b>LINE2</b> | <b>EAX= FFFFFFFE6h</b>                      | <b>Negate register EAX with two's complement</b>           |
| <b>LINE3</b> | <b>EBX= 0000001Eh<br/>Yval= 0000001Eh</b>   | <b>Move the value of Yval (30d) into register EBX</b>      |
| <b>LINE4</b> | <b>EBX= FFFFFFFF6h<br/>Zval= 00000028h</b>  | <b>Subtract the value of Zval (40d) from register EBX</b>  |
| <b>LINE5</b> | <b>EAX= FFFFFFFDCh<br/>EBX= FFFFFFFF6h</b>  | <b>Add value of register EBX to register EAX</b>           |
| <b>LINE6</b> | <b>EAX= FFFFFFFDDh</b>                      | <b>Increment of EAX by 1</b>                               |
| <b>LINE7</b> | <b>EAX= FFFFFFFDDh<br/>Rval= FFFFFFFDDh</b> | <b>Move value store in register EAX into variable Rval</b> |

b) Paste here screenshot of all registers' content after each LINE is executed:

LINE1:

Registers

EAX = 0000001A EBX = 00CF2000 ECX = 00641005  
EDX = 00641005 ESI = 00641005 EDI = 00641005  
EIP = 00641015 ESP = 00BFF96C EBP = 00BFF978  
EFL = 00000246

LINE2:

Registers

EAX = FFFFFFFE6 EBX = 00CF2000 ECX = 00641005  
EDX = 00641005 ESI = 00641005 EDI = 00641005  
EIP = 00641017 ESP = 00BFF96C EBP = 00BFF978  
EFL = 00000293

0x00644008 = 0000001E

LINE3:

Registers

EAX = FFFFFFFE6 EBX = 0000001E ECX = 00641005  
EDX = 00641005 ESI = 00641005 EDI = 00641005  
EIP = 0064101D ESP = 00BFF96C EBP = 00BFF978  
EFL = 00000293

0x0064400C = 00000028

LINE4:

Registers

EAX = FFFFFFFE6 EBX = FFFFFFF6 ECX = 00641005  
EDX = 00641005 ESI = 00641005 EDI = 00641005  
EIP = 00641023 ESP = 00BFF96C EBP = 00BFF978  
EFL = 00000287

LINE5:

Registers

EAX = FFFFFFFDC EBX = FFFFFFF6 ECX = 00641005  
EDX = 00641005 ESI = 00641005 EDI = 00641005  
EIP = 00641025 ESP = 00BFF96C EBP = 00BFF978  
EFL = 00000283

LINE6:

Registers

EAX = FFFFFFFDD EBX = FFFFFFF6 ECX = 00641005  
EDX = 00641005 ESI = 00641005 EDI = 00641005  
EIP = 00641026 ESP = 00BFF96C EBP = 00BFF978  
EFL = 00000287

0x00644000 = 00000000

LINE7:

Registers

EAX = FFFFFFFDD EBX = FFFFFFF6 ECX = 00641005  
EDX = 00641005 ESI = 00641005 EDI = 00641005  
EIP = 0064102B ESP = 00BFF96C EBP = 00BFF978  
EFL = 00000287

Q3. Execute the program below. Determine output of the program by inspecting the content of the related registers.

- a) Fill in Table 3 with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.
- b) Paste the screenshot of all registers' content after each LINE is executed.

**Arithmetic expression:  $\text{var4} = [(\text{var1} * \text{var2}) + \text{var3}] - 1$**

```
include irvine32.inc

.data
var1 DWORD 5           ;00000005h
var2 DWORD 10          ;0000000Ah
var3 DWORD 20          ;00000014h
var4 DWORD ?

.code
main proc
    mov eax, var1       ; LINE1
    mul var2            ; LINE2
    add eax, var3       ; LINE3
    dec eax             ; LINE4
    exit
main endp
end main
```

### **Answer Q3**

- a) Fill (Write) in the contents for the related register in each line:

**Table 3**

|       |                                                   |                                                                 |
|-------|---------------------------------------------------|-----------------------------------------------------------------|
| LINE1 | <b>EAX = 00000005h</b><br><b>var1 = 00000005h</b> | Move the value of var1 (5d) into register EAX                   |
| LINE2 | <b>EAX = 00000032h</b><br><b>var2 = 0000000Ah</b> | Multiply the value of var2 (10d) into register EAX              |
| LINE3 | <b>EAX = 00000046h</b><br><b>var3 = 00000014h</b> | Add the value of var3 (20d) into register EAX                   |
| LINE4 | <b>EAX = 00000045h</b><br><b>var4 = 00000045h</b> | Decrease 1d in EAX and move the value of EAX into variable var4 |

b) Paste here screenshot of all registers' content after each LINE is executed:

LINE1:

| Memory 1                                                                                                                                                 | Registers | Autos | Watch 1 |
|----------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|-------|---------|
| EAX = 00000005 EBX = 00CC0000 ECX = 00A11005 EDX = 00A11005 ESI = 00A11005<br>EDI = 00A11005 EIP = 00A11015 ESP = 00BFFBB0 EBP = 00BFFBBC EFL = 00000246 |           |       |         |
| 0x00A14004 = 0000000A                                                                                                                                    |           |       |         |

LINE2:

| Memory 1                                                                                                                                                 | Registers | Autos | Watch 1 |
|----------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|-------|---------|
| EAX = 00000032 EBX = 00CC0000 ECX = 00A11005 EDX = 00000000 ESI = 00A11005<br>EDI = 00A11005 EIP = 00A1101B ESP = 00BFFBB0 EBP = 00BFFBBC EFL = 00000202 |           |       |         |
| 0x00A14008 = 00000014                                                                                                                                    |           |       |         |

LINE3:

| Memory 1                                                                                                                                                 | Registers | Autos | Watch 1 |
|----------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|-------|---------|
| EAX = 00000046 EBX = 00CC0000 ECX = 00A11005 EDX = 00000000 ESI = 00A11005<br>EDI = 00A11005 EIP = 00A11021 ESP = 00BFFBB0 EBP = 00BFFBBC EFL = 00000202 |           |       |         |

LINE4:

| Memory 1                                                                                                                                                 | Registers | Autos | Watch 1 |
|----------------------------------------------------------------------------------------------------------------------------------------------------------|-----------|-------|---------|
| EAX = 00000045 EBX = 00CC0000 ECX = 00A11005 EDX = 00000000 ESI = 00A11005<br>EDI = 00A11005 EIP = 00A11022 ESP = 00BFFBB0 EBP = 00BFFBBC EFL = 00000202 |           |       |         |

Q4. Execute the program below. Determine output of the program by inspecting the content of the related registers.

- Fill in Table 4 with the content of each register or variable on every LINE, in Hexadecimal (as per the output). Please complete the comments for every LINE.
- Paste the screenshot of all registers' content after each LINE is executed.

**Arithmetic expression:  $\text{var4} = (\text{var1} * 5) / (\text{var2} - 3)$**

```
include irvine32.inc
.data
    var1 WORD 40
    var2 WORD 10
    var4 WORD ?
.code
main proc
    mov ax,var1           ; LINE1
    mov bx,5              ; LINE2
    mul bx                ; LINE3
    mov bx,var2           ; LINE4
    sub bx,3              ; LINE5
    div bx                ; LINE6
    mov var4,ax           ; LINE7
    exit
main endp
end main
```

#### Answer Q4

- Fill (Write) in the contents for the related register in each line:

**Table 4**

|              |                                             |                                                                                                                                |
|--------------|---------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| <b>LINE1</b> | <b>AX= 0028h<br/>var1= 0028h</b>            | <b>Move the value of var1 (40d) into the register AX</b>                                                                       |
| <b>LINE2</b> | <b>BX= 0005h</b>                            | <b>Move the immediate value (5d) into register BX</b>                                                                          |
| <b>LINE3</b> | <b>AX= 00C8h<br/>BX= 0005h</b>              | <b>Multiply the value of register BX to register AX and store the value in register DX:AX</b>                                  |
| <b>LINE4</b> | <b>BX= 000Ah<br/>var2= 000Ah</b>            | <b>Move the value of var2 (10d) into register BX</b>                                                                           |
| <b>LINE5</b> | <b>BX= 0007h</b>                            | <b>Subtract the value in register BX with an immediate value (3d)</b>                                                          |
| <b>LINE6</b> | <b>AX= 001Ch<br/>BX= 0007h<br/>DX=0004h</b> | <b>Divide the register AX by register BX, the quotient is then store in register AX and the remainder store in register DX</b> |
| <b>LINE7</b> | <b>AX= 001Ch<br/>var4= 001Ch</b>            | <b>Move the value of register AX into variable var4</b>                                                                        |

b) Paste here screenshot of all registers' content after each LINE is executed:

LINE1:

Registers

EAX = 00EF0028 EBX = 00C22000 ECX = 00AB1005  
EDX = 00AB1005 ESI = 00AB1005 EDI = 00AB1005  
EIP = 00AB1016 ESP = 00EFF91C EBP = 00EFF928  
EFL = 00000246

LINE2:

Registers

EAX = 00EF0028 EBX = 00C20005 ECX = 00AB1005  
EDX = 00AB1005 ESI = 00AB1005 EDI = 00AB1005  
EIP = 00AB101A ESP = 00EFF91C EBP = 00EFF928  
EFL = 00000246

LINE3:

Registers

EAX = 00EF00C8 EBX = 00C20005 ECX = 00AB1005  
EDX = 00AB0000 ESI = 00AB1005 EDI = 00AB1005  
EIP = 00AB101D ESP = 00EFF91C EBP = 00EFF928  
EFL = 00000246

LINE4:

Registers

EAX = 00EF00C8 EBX = 00C2000A ECX = 00AB1005  
EDX = 00AB0000 ESI = 00AB1005 EDI = 00AB1005  
EIP = 00AB1024 ESP = 00EFF91C EBP = 00EFF928  
EFL = 00000246

LINE5:

Registers

EAX = 00EF00C8 EBX = 00C20007 ECX = 00AB1005  
EDX = 00AB0000 ESI = 00AB1005 EDI = 00AB1005  
EIP = 00AB1028 ESP = 00EFF91C EBP = 00EFF928  
EFL = 00000202

LINE6:

Registers

EAX = 00EF001C EBX = 00C20007 ECX = 00AB1005  
EDX = 00AB0004 ESI = 00AB1005 EDI = 00AB1005  
EIP = 00AB102B ESP = 00EFF91C EBP = 00EFF928  
EFL = 00000212

LINE7:

Registers

EAX = 00EF001C EBX = 00C20007 ECX = 00AB1005  
EDX = 00AB0004 ESI = 00AB1005 EDI = 00AB1005  
EIP = 00AB1031 ESP = 00EFF91C EBP = 00EFF928  
EFL = 00000212

## **Short Notes for MUL CX and DIV BL:**

### **MUL CX**

- a. MUL always uses AX (or its extended versions EAX or RAX) as the implicit destination register.
- b. The operand size determines the size of the result:
  - i. Byte-sized operand: Result in AX
  - ii. Word-sized operand: Result in DX:AX
  - iii. Doubleword-sized operand (32-bit mode): Result in EDX:EAX
  - iv. Quadword-sized operand (64-bit mode): Result in RDX:RAX
- c. The upper half of the result (DX or EDX or RDX) holds any overflow bits.
- d. The Carry Flag (CF) is set if the upper half of the product is non-zero.

### **DIV BL**

- a. DIV always uses the DX:AX or EDX:EAX pair as the implicit dividend register.
  - b. The divisor is specified as the operand of the DIV instruction.
  - c. The quotient is stored in AX (for 16-bit division) or EAX (for 32-bit division).
  - d. The remainder is stored in DX.
  - e. Clear DX (or EDX for 32-bit division) before division to ensure a correct 16-bit or 32-bit dividend.
  - f. If the divisor is 0, a division error occurs.
  - g. The Overflow Flag (OF) is set if the quotient is too large to fit in the destination register.
-

Q5. Given the following instructions as is Code Snippet 1.

- a) Write a full program to execute the Code Snippet 1.
- b) What are the contents of the related registers after Code Snippet 1 is executed? Paste the screenshot of DumpReg.

**; Code Snippet 1 (MUL CX)**

```
MOV DX, 0      ; Clear DX
MOV AX, 1000h   ; Load 1000h into AX
MOV CX, 25h     ; Load 25h into CX
MUL CX          ; Multiply AX by CX, storing the result in DX:AX
```

**Answer Q5**

- a) Screenshot of full program (.asm) :

```
1  TITLE lab2
2
3  ; Author: Lim Yu Han
4  ; Date: 03 June 2024
5
6
7  include Irvine32.inc
8
9  .code
10 main proc
11     MOV DX, 0      ; Clear DX
12     MOV AX, 1000h   ; Load 1000h into AX
13     MOV CX, 25h     ; Load 25h into CX
14     MUL CX          ; Multiply AX by CX, storing the result in DX : AX
15
16     call DumpRegs   ; display the register
17     exit
18
19 main ENDP
20 END main
```

- b) Paste here the screenshot of the final registers' content (DumpReg):

```
EAX=008F5000  EBX=00605000  ECX=00530025  EDX=00530002
ESI=005310AA  EDI=005310AA  EBP=008FFC58  ESP=008FFC4C
EIP=00533674  EFL=00000A07  CF=1  SF=0  ZF=0  OF=1  AF=0  PF=1
```

Memory 1    Registers    Autos    Watch 1

```
EAX = 008F5000  EBX = 00605000  ECX = 00530025  EDX = 00530002  ESI = 005310AA
EDI = 005310AA  EIP = 00533674  ESP = 008FFC4C  EBP = 008FFC58  EFL = 00000A07
```

Q6. Given the following instructions as is Code Snippet 2.

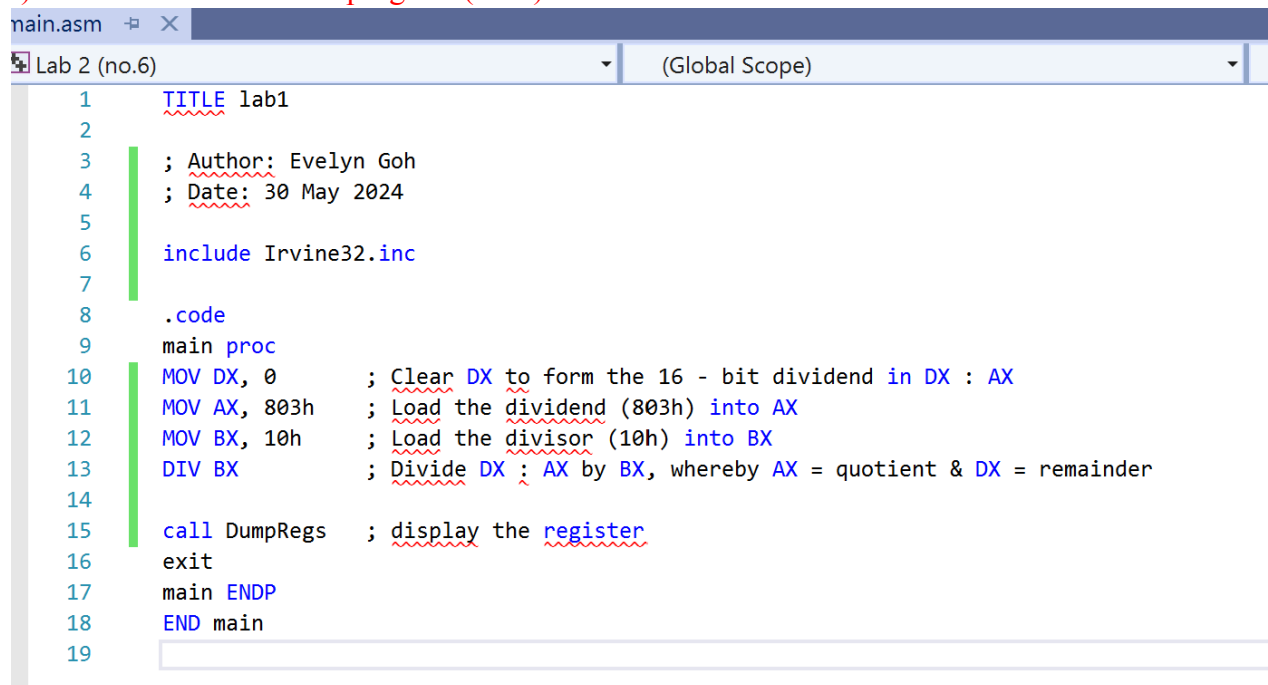
- Write a full program to execute the Code Snippet 2.
- What are the contents of the related registers after Code Snippet 2 is executed? Paste the screenshot of DumpReg.

**; Code Snippet 2 (DIV BL)**

```
MOV DX, 0      ; Clear DX to form the 16-bit dividend in DX:AX
MOV AX, 803h   ; Load the dividend (8003h) into AX
MOV BL, 10h    ; Load the divisor (10h) into BL
DIV BL         ; Divide DX:AX by BL, whereby AX=quotient & DX=remainder
```

**Answer Q6**

- Screenshot of full program (.asm) :



```
nain.asm  X
Lab 2 (no.6)  (Global Scope)
1  TITLE lab1
2
3  ; Author: Evelyn Goh
4  ; Date: 30 May 2024
5
6  include Irvine32.inc
7
8  .code
9  main proc
10 MOV DX, 0      ; Clear DX to form the 16 - bit dividend in DX : AX
11 MOV AX, 803h   ; Load the dividend (803h) into AX
12 MOV BX, 10h    ; Load the divisor (10h) into BX
13 DIV BX        ; Divide DX : AX by BX, whereby AX = quotient & DX = remainder
14
15 call DumpRegs  ; display the register
16 exit
17 main ENDP
18 END main
19
```

- Paste here the screenshot of the final registers' content (DumpReg):

```
EAX=00CF0080  EBX=00A10010  ECX=00D810AA  EDX=00D80003
ESI=00D810AA  EDI=00D810AA  EBP=00CFF924  ESP=00CFF918
EIP=00D83674  EFL=00000212  CF=0  SF=0  ZF=0  OF=0  AF=1  PF=0
```

**Registers**

```
EAX = 00D30080 EBX = 00B70010 ECX = 00D810AA
EDX = 00D80003 ESI = 00D810AA EDI = 00D810AA
EIP = 00D83674 ESP = 00D3FE9C EBP = 00D3FEA8
EFL = 00000212
```